



SRI RAMAKRISHNA
ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.



www.srec.ac.in
+917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ALM

TEMPERATURE MONITORING SYSTEM USING LPC2148

AJAY IYANRAJ (71812401007)
AKASHVARMAN R (71812401008)
AKHILESH RAJ P (71812401009)
AKSHAY KRISHNA S M (71812401010)
AKSHAYA KEERTHI S (71812401011)

THIRD YEAR B.E. CSE – V SEM

Academic Year 2026-2027

20EC252 & MICROCONTROLLER AND PROGRAMMING

COURSE COORDINATOR

Mrs.M.Amuthasurabi,AP/CSE



SRI RAMAKRISHNA
ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.



www.srec.ac.in
+917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20EC252 & MICROCONTROLLER AND PROGRAMMING LABORATORY

TEMPERATURE MONITORING SYSTEM USING LPC2148

Assessment

INDEX

S. No.	Name & Roll No.	Introduction (2)	Logic Building (2)	Implementation Steps (4)	Report (2)	Total (10)
1.	AJAY IYANRAJ 71812401007					
2.	AKASHVARMAN R 71812401008					
3.	AKHILESH RAJ P 71812401009					
4.	AKSHAY KRISHNA S M 71812401010					
5.	AKSHAYA KEERTHI S 71812401011					

SIGNATURE OF COURSE COORDINATOR
Mrs.M.Amuthasurabi,AP/CSE



CONTENTS

S. No.	TITLE	PAGE NUMBER
	TEMPERATURE MONITORING SYSTEM USING LPC2148	4
1.	INTRODUCTION	5
2.	WORKFLOW DESCRIPTION	7
3.	IMPLEMENTATION STEPS	9
4.	CODE IMPLEMENTATION	11
5.	OUTPUT	14
6.	DIAGRAMMATIC REPRESENTATION	18
7.	CONCLUSION	20

TEMPERATURE MONITORING SYSTEM USING LPC2148

Abstract

Temperature is one of the most commonly monitored physical quantities in industries, laboratories, server rooms, greenhouses and homes. This project designs a digital temperature monitoring system around the LPC2148 ARM7TDMI-S microcontroller. An LM35 precision temperature sensor produces an analog voltage of 10 mV for every degree Celsius. This voltage is applied to the on-chip 10-bit Analog to Digital Converter (ADC0, channel AD0.1 on pin P0.28) of the LPC2148, which converts it into a digital number between 0 and 1023. The program converts the number into degrees Celsius, shows it on a 16 x 2 LCD, sends it to a PC through UART0, and switches a fan, a buzzer and an alarm LED when the temperature crosses the limit of 40 °C. The report explains in detail how the ADC of the LPC2148 is configured and used to read the sensor value.

Working in brief

The LM35 output voltage is connected to the analog input AD0.1 (P0.28). The ADC of the LPC2148 converts it into a number from 0 to 1023 in about 3 μ s. The program averages eight conversions, calculates the temperature, shows it on the LCD and the serial terminal, and drives the fan, the buzzer and the alarm LED when the temperature reaches 40 °C. The fan is switched off again at 38 °C so that it does not switch on and off continuously.

Key formulas

Quantity	Formula
Input voltage	$V_{in} = \text{ADC} \times 3.3 / 1023$
Temperature	$T (^{\circ}\text{C}) = V_{in} / 0.010 = \text{ADC} \times 330 / 1023$
ADC result	$\text{AD0DR1 bits 15:6} = (V_{in} / V_{REF}) \times 1023$

Objectives

- To study the architecture and the on-chip ADC of the LPC2148 microcontroller.
- To interface the LM35 analog temperature sensor with the ADC channel AD0.1 (P0.28).
- To convert the digital ADC result into the temperature in degrees Celsius and display it on an LCD and a serial terminal.
- To switch a fan, a buzzer and an alarm LED automatically when the temperature exceeds the set limit.

Hardware and software used

Component / Tool	Specification	Purpose
LPC2148 development board	ARM7TDMI-S, 512 KB flash, 12 MHz crystal	Main controller with on-chip ADC
LM35 sensor	10 mV / °C, 4 V to 30 V supply	Converts temperature into voltage
16 x 2 LCD (JHD162A)	HD44780 controller, 4-bit mode	Shows temperature and status
Relay + DC fan, buzzer, LED	5 V relay, 5 V buzzer, 330 Ω resistor	Cooling and alarm outputs
Keil uVision, Flash Magic	ARM compiler and ISP programmer	Writing, building and loading the code
Serial terminal / HTML simulator	9600 baud, 8N1	Viewing the output

1. INTRODUCTION

1.1 Background

Many systems must run within a safe temperature range. A server room that becomes too hot damages the equipment, a cold storage that becomes warm spoils the food, and an overheated motor may burn. A temperature monitoring system measures the temperature continuously, shows it to the user and takes an action, such as switching on a fan, when the temperature is high. A microcontroller based system is small, cheap and accurate, and it can also send the readings to a computer.

1.2 Problem statement

Design a temperature monitoring system using LPC2148. Explain how the ADC (Analog to Digital Converter) of LPC2148 can be used to read the temperature sensor value.

1.3 LPC2148 microcontroller

The LPC2148 is a 16/32-bit ARM7TDMI-S based microcontroller from NXP (Philips) with real-time emulation and embedded trace support. It is widely used in embedded system laboratories because it contains many peripherals on a single chip.

Feature	Details
CPU	16/32-bit ARM7TDMI-S, up to 60 MHz using the on-chip PLL
Memory	512 KB on-chip flash, 32 KB static RAM + 8 KB USB RAM
ADC	Two 10-bit ADCs: ADC0 with 6 channels and ADC1 with 8 channels
DAC	One 10-bit digital to analog converter (AOUT on P0.25)
Serial	Two UARTs, two I2C, SPI, SSP, USB 2.0 full speed device
Timers	Two 32-bit timers, PWM unit, real time clock, watchdog timer
I/O	Up to 45 general purpose I/O pins, 5 V tolerant digital pins
Supply	3.3 V (core 1.8 V from an internal regulator)

1.4 ADC of the LPC2148

The ADC of the LPC2148 is a successive approximation converter. It has a 10-bit resolution, so the input voltage from 0 V to VREF is divided into 1024 steps. The important features of ADC0 are given below.

Parameter	Value for ADC0
Resolution	10 bit (0 to 1023 counts)
Input channels	AD0.1, AD0.2, AD0.3, AD0.4, AD0.6 and AD0.7 (pins P0.28, P0.29, P0.30, P0.25, P0.4, P0.5)
Reference voltage	VREF = 3.3 V (VDDA); input range 0 V to 3.3 V
Maximum ADC clock	4.5 MHz (obtained from PCLK using the CLKDIV field of AD0CR)
Conversion time	11 clock cycles for 10 bits = 2.44 μ s at 4.5 MHz
Conversion modes	Software controlled (one channel) and burst mode (several channels repeatedly)
Start options	Software start, external pin edge or timer match
Result registers	AD0DR0 to AD0DR7 for each channel and AD0GDR global data register

1.5 LM35 temperature sensor

Parameter	Value
Type	Precision integrated-circuit temperature sensor (analog output)
Scale factor	+10.0 mV per °C (linear)
Temperature range	0 °C to 100 °C (LM35D) / -55 °C to 150 °C (LM35)
Accuracy	±0.5 °C at 25 °C
Supply voltage	4 V to 30 V (5 V used in this project)
Output at 25 °C	250 mV
Pins	+Vs, Vout, GND (TO-92 package)

1.6 Why the ADC is needed

The LM35 gives an analog voltage but the processor of the LPC2148 can process only digital numbers. The ADC bridges the gap: it samples the sensor voltage and converts it into a 10-bit number which the program can calculate, compare, display and transmit. Without the ADC the microcontroller could not measure the temperature.

1.7 Comparison of temperature sensors

Sensor	Output	Range	Advantage	Disadvantage
LM35	Analog, 10 mV / °C	-55 to 150 °C	Linear, no calibration	Needs an ADC
Thermistor (NTC)	Resistance	-40 to 125 °C	Very cheap	Non-linear
DS18B20	Digital (1-Wire)	-55 to 125 °C	No ADC needed	Slower, needs a library
Thermocouple (K)	Millivolts	-200 to 1350 °C	Very wide range	Needs an amplifier

1.8 Applications

- Server rooms and electrical panels - automatic fan control to avoid overheating.
- Cold storage, incubators and greenhouses - continuous temperature display and alarm.
- Industrial motors and transformers - warning before the temperature becomes dangerous.
- Home appliances such as water heaters and room coolers.

1.9 Design specifications

Parameter	Specification
Measuring range	0 °C to 100 °C (Vout = 0 V to 1.0 V)
Resolution	0.32 °C per ADC count
Sampling	8 conversions averaged, one reading every 0.5 s
Alarm limit	40 °C ON, 38 °C OFF (hysteresis 2 °C)
Display	16 x 2 LCD and UART0 at 9600 baud
Supply	5 V for LM35, LCD and relay; 3.3 V for LPC2148

2. WORKFLOW DESCRIPTION

2.1 Overall working of the system

- Step 1: Sensing:** The LM35 senses the surrounding temperature and gives an output voltage $V_{out} = 10 \text{ mV} \times T \text{ (in } ^\circ\text{C)}$.
- Step 2: Analog input:** V_{out} is connected to the pin P0.28 which is configured as the analog input AD0.1 of ADC0.
- Step 3: Conversion:** The software starts a conversion; the ADC samples V_{out} and the successive approximation logic produces a 10-bit result in AD0DR1.
- Step 4: Reading:** The program waits until the DONE bit (bit 31) of AD0DR1 becomes 1 and then reads bits 15:6 as the ADC value.
- Step 5: Calculation:** Eight conversions are averaged to reduce noise and the temperature is calculated as $T = \text{ADC} \times 330 / 1023$.
- Step 6: Display:** The temperature and the ADC value are displayed on the 16 x 2 LCD and sent to the PC through UART0.
- Step 7: Control:** If T is 40°C or more, the fan, the buzzer and the alarm LED are switched ON; they are switched OFF when T falls to 38°C or less.
- Step 8: Repeat:** The program waits for 500 ms and repeats the cycle for ever.

2.2 Working principle of the successive approximation ADC

The ADC compares the sampled input voltage with a voltage produced by an internal 10-bit digital to analog converter. Starting from the most significant bit, each bit is set to 1 and kept only if the DAC voltage is still not greater than the input. After 10 comparisons the bits that were kept form the digital result. The result is proportional to the input: $\text{ADC value} = (V_{in} / V_{REF}) \times 1023$.

2.3 Conversion formulas

Quantity	Formula	Example (ADC = 140)
Input voltage	$V_{in} = \text{ADC} \times V_{REF} / 1023$	$140 \times 3.3 / 1023 = 0.4516 \text{ V}$
Temperature	$T = V_{in} / 0.010 = \text{ADC} \times 330 / 1023$	$140 \times 330 / 1023 = 45.2^\circ\text{C}$
Resolution (1 count)	$V_{REF} / 1023 = 3.226 \text{ mV}$	$3.226 \text{ mV} / 10 \text{ mV per } ^\circ\text{C} = 0.32^\circ\text{C}$
ADC value from voltage	$\text{ADC} = V_{in} \times 1023 / V_{REF}$	$0.450 \text{ V} \times 1023 / 3.3 = 139.5 = 140$
ADC clock	$\text{PCLK} / (\text{CLKDIV} + 1)$	$15 \text{ MHz} / 4 = 3.75 \text{ MHz (below } 4.5 \text{ MHz)}$
Conversion time	$11 / \text{ADC clock}$	$11 / 3.75 \text{ MHz} = 2.93 \mu\text{s}$

2.4 ADC control register AD0CR (0x01200302 is used)

Bits	Field	Value used	Meaning
7:0	SEL	0000 0010	Selects channel AD0.1 (bit 1 = 1)
15:8	CLKDIV	0000 0011	ADC clock = PCLK / (3 + 1) = 3.75 MHz
16	BURST	0	Software controlled conversion (no burst)
19:17	CLKS	000	11 clocks, 10 bit accuracy
21	PDN	1	ADC is operational (not in power-down)
26:24	START	001	Start the conversion now
27	EDGE	0	Not used (only for pin / timer start)

2.5 ADC data register AD0DR1

Bits	Field	Meaning
5:0	-	Reserved
15:6	V/VREF	10-bit result: (Vin / VREF) x 1023 (stored in the upper bits)
26:24	CHN	Channel number from which the result was converted (001 = AD0.1)
30	OVERRUN	Set when a result is lost in burst mode
31	DONE	Set to 1 when the conversion is complete; cleared when AD0DR1 is read

2.6 Pin connections

Signal	LPC2148 pin	Connected to
Sensor output	P0.28 (AD0.1)	Vout of LM35
LCD data D4 - D7	P1.16 - P1.19	LCD pins 11 - 14
LCD RS, EN	P0.10, P0.11	LCD pins 4 and 6 (RW and VSS to GND)
Fan relay	P0.8	Relay driver and DC fan
Buzzer	P0.7	Buzzer through BC547 transistor
Alarm LED	P0.9	LED with 330 Ω resistor
UART0 TXD0 / RXD0	P0.0 / P0.1	USB to serial converter (PC)

2.7 Fan, buzzer and LED control (with hysteresis)

Temperature (T)	Previous state	New state	Reason
T >= 40 °C	OFF or ON	ON	Temperature is at or above the limit
38 °C < T < 40 °C	OFF	OFF	Not yet at the limit
38 °C < T < 40 °C	ON	ON	Hysteresis: stays ON to avoid rapid switching
T <= 38 °C	ON or OFF	OFF	Temperature has fallen enough

3. IMPLEMENTATION STEPS

3.1 Steps followed

- Step 1:** **Understand the sensor:** LM35 gives 10 mV per °C; the maximum temperature to be measured is 100 °C = 1.0 V, well below VREF (3.3 V).
- Step 2:** **Make the connections:** connect Vout of LM35 to P0.28, power the LM35 from 5 V, and connect the LCD, fan relay, buzzer, LED and the UART as in Section 2.6.
- Step 3:** **Set the clock:** configure the PLL to get CCLK = 60 MHz from the 12 MHz crystal and keep VPBDIV = 0 so that PCLK = 15 MHz.
- Step 4:** **Select the pin function:** write PINSEL1 |= (1 << 24) so that P0.28 works as the analog input AD0.1.
- Step 5:** **Configure the ADC:** write AD0CR with SEL = 0x02, CLKDIV = 3, PDN = 1 and START = 1; the ADC clock is 3.75 MHz.
- Step 6:** **Read the result:** poll bit 31 (DONE) of AD0DR1 and take bits 15:6 as the 10-bit digital value.
- Step 7:** **Reduce noise:** take 8 conversions and use their average.
- Step 8:** **Convert to temperature:** temperature = ADC value x 330 / 1023 degrees Celsius.
- Step 9:** **Show and send the result:** display on the 16 x 2 LCD in 4-bit mode and transmit the same line through UART0 at 9600 baud.
- Step 10:** **Control the outputs:** switch ON the fan, buzzer and LED at 40 °C and switch OFF at 38 °C (2 °C hysteresis).
- Step 11:** **Compile and load:** build main.c in Keil uVision, create the HEX file and load it into the flash using Flash Magic.
- Step 12:** **Test:** heat and cool the sensor, compare the readings with a reference thermometer and verify the alarm action.

3.2 Important register settings

Register	Value	Purpose
PLL0CFG	0x24	M = 5, P = 2: 12 MHz x 5 = 60 MHz CCLK
VPBDIV	0x00	PCLK = CCLK / 4 = 15 MHz
PINSEL1	0x01000000	P0.28 selected as AD0.1
AD0CR	0x01200302	Channel 1, 3.75 MHz clock, ADC ON, start conversion
IO0DIR	(1<<7) (1<<8) (1<<9) (1<<10) (1<<11)	Buzzer, fan, LED, LCD RS and EN as outputs
IO1DIR	0x000F0000	P1.16 - P1.19 as LCD data lines
U0LCR / U0DLL	0x83 / 98	8N1 and 9600 baud from PCLK = 15 MHz

3.3 Expected ADC readings of the LM35

Temp (°C)	LM35 (mV)	Vin (V)	ADC (dec)	ADC (hex)	ADC (binary)
0	0	0.000	0	0x000	0000000000
10	100	0.100	31	0x01F	0000011111
25	250	0.250	78	0x04E	0001001110
30	300	0.300	93	0x05D	0001011101
40	400	0.400	124	0x07C	0001111100
50	500	0.500	155	0x09B	0010011011
75	750	0.750	233	0x0E9	0011101001
100	1000	1.000	310	0x136	0100110110

3.4 Calibration and testing

Reference thermometer (°C)	ADC value	Displayed (°C)	Error (°C)	Fan
25.0	78	25.2	+0.2	OFF
30.0	93	30.0	0.0	OFF
35.0	109	35.2	+0.2	OFF
40.0	124	40.0	0.0	ON
45.0	140	45.2	+0.2	ON

3.5 Precautions

- Keep the analog input below VREF (3.3 V); the LM35 output is only 1.0 V at 100 °C.
- Use a stable 3.3 V for VREF / VDDA and connect VSSA to the common ground.
- Keep the sensor wires short or use a shielded cable, and add a 0.1 μ F capacitor near the ADC pin.
- Do not exceed the ADC clock of 4.5 MHz; use CLKDIV = 3 with PCLK = 15 MHz.

3.6 Troubleshooting

Problem	Possible cause	Remedy
ADC value always 0	P0.28 not set as AD0.1	Set bit 24 of PINSEL1
Reading does not change	PDN bit not set / conversion not started	Use AD0CR = 0x01200302
Temperature fluctuates	Electrical noise	Average 8 samples; add capacitor
LCD shows garbage	Wrong 4-bit initialisation	Send 0x02 then 0x28 with proper delays

3.7 Timing of one measurement cycle

- One ADC conversion takes 11 clocks at 3.75 MHz = 2.93 μ s; eight conversions for averaging take about 23.5 μ s.
- Updating the LCD takes about 0.1 s and sending one 38-character line at 9600 baud takes about 40 ms.
- With the 500 ms delay, the temperature is measured and displayed about every 0.65 s.

4. CODE IMPLEMENTATION

The embedded C program (main.c) is written for the Keil uVision compiler using the header file lpc214x.h. It contains the functions for the PLL, the UART, the LCD and the ADC, followed by the main loop.

Program: main.c

```
/* Temperature Monitoring System using LPC2148 (ADC0 channel 1, LM35 on P0.28)
   Display : 16x2 LCD (4-bit) D4-D7 = P1.16-P1.19, RS = P0.10, EN = P0.11
   Outputs : FAN relay = P0.8, BUZZER = P0.7, ALARM LED = P0.9
   Serial : UART0 9600 baud, 8N1 (P0.0 = TXD0, P0.1 = RXD0)
   Clock : Fosc = 12 MHz, CCLK = 60 MHz, PCLK = 15 MHz */

#include <lpc214x.h>
#include <stdio.h>

#define LCD_DATA (0x0Fu << 16) /* P1.16 - P1.19 -> D4 - D7 */
#define LCD_RS (1u << 10) /* P0.10 */
#define LCD_EN (1u << 11) /* P0.11 */
#define BUZZER (1u << 7) /* P0.7 */
#define FAN (1u << 8) /* P0.8 */
#define ALARM_LED (1u << 9) /* P0.9 */

#define TEMP_LIMIT 40.0f /* fan ON at or above 40 deg C */
#define HYSTERESIS 2.0f /* fan OFF at or below 38 deg C */
#define SAMPLES 8 /* conversions averaged per reading */

/* ----- delay ----- */
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for (i = 0; i < ms; i++)
        for (j = 0; j < 6000; j++); /* about 1 ms at CCLK = 60 MHz */
}

/* ----- PLL : 12 MHz -> 60 MHz ----- */
void pll_init(void)
{
    PLL0CFG = 0x24; /* M = 5, P = 2 */
    PLL0CON = 0x01; PLL0FEED = 0xAA; PLL0FEED = 0x55;
    while (!(PLL0STAT & (1 << 10))); /* wait for PLL lock */
    PLL0CON = 0x03; PLL0FEED = 0xAA; PLL0FEED = 0x55; /* enable + connect */
    VPDIV = 0x00; /* PCLK = CCLK / 4 = 15 MHz */
}

/* ----- UART0 : 9600 baud ----- */
void uart0_init(void)
{
    PINSEL0 |= 0x00000005; /* P0.0 = TXD0, P0.1 = RXD0 */
    U0LCR = 0x83; /* 8 bits, no parity, 1 stop, DLAB=1 */
    U0DLL = 98; U0DLM = 0; /* 15 MHz / (16 x 9600) = 97.6 */
    U0LCR = 0x03; /* DLAB = 0 */
}

void uart0_puts(const char *s)
{
    while (*s) {
        while (!(U0LSR & 0x20)); /* wait until THR is empty */
        U0THR = *s++;
    }
}

/* ----- 16x2 LCD (4-bit mode) ----- */
void lcd_write_nibble(unsigned char n)
{
    IO1CLR = LCD_DATA;
    IO1SET = ((unsigned int)n & 0x0F) << 16;
    IO0SET = LCD_EN; delay_ms(1); IO0CLR = LCD_EN;
}

void lcd_cmd(unsigned char c)
{
    IO0CLR = LCD_RS;
    lcd_write_nibble(c >> 4); lcd_write_nibble(c);
    delay_ms(2);
}
```

```

void lcd_data(unsigned char d)
{
    IO0SET = LCD_RS;
    lcd_write_nibble(d >> 4); lcd_write_nibble(d);
    delay_ms(2);
}

void lcd_puts(const char *s) { while (*s) lcd_data(*s++); }

void lcd_init(void)
{
    IO0DIR |= LCD_RS | LCD_EN;
    IO1DIR |= LCD_DATA;
    delay_ms(20);
    lcd_cmd(0x02);          /* 4-bit mode */
    lcd_cmd(0x28);          /* 2 lines, 5x7 font */
    lcd_cmd(0x0C);          /* display ON, cursor OFF */
    lcd_cmd(0x06);          /* auto increment */
    lcd_cmd(0x01);          /* clear display */
}

/* ----- ADC0 channel 1 (AD0.1 on P0.28) ----- */
void adc_init(void)
{
    PINSEL1 |= (1u << 24); /* P0.28 -> AD0.1 (bits 25:24 = 01) */
}

unsigned int adc_read(void)
{
    unsigned int val;
    /* SEL = channel 1, CLKDIV = 3 (15 MHz / 4 = 3.75 MHz), PDN = 1, START = 001 */
    AD0CR = (1u << 1) | (3u << 8) | (1u << 21) | (1u << 24);
    do { val = AD0DR1; } while (!(val & (1u << 31))); /* wait for DONE bit */
    AD0CR &= ~(7u << 24); /* stop conversion */
    return (val >> 6) & 0x3FF; /* 10-bit result in bits 15:6 */
}

unsigned int adc_average(void)
{
    unsigned int i, sum = 0;
    for (i = 0; i < SAMPLES; i++) sum += adc_read();
    return sum / SAMPLES;
}

/* ----- main ----- */
int main(void)
{
    unsigned int adc;
    float temp;
    char line[32];
    unsigned char fan_on = 0;

    pll_init();
    IO0DIR |= BUZZER | FAN | ALARM_LED;
    IO0CLR = BUZZER | FAN | ALARM_LED;
    uart0_init();
    lcd_init();
    adc_init();

    lcd_puts("LPC2148 TEMP MON");
    uart0_puts("\r\nLPC2148 Temperature Monitoring System\r\n");
    delay_ms(1500);

    while (1)
    {
        adc = adc_average();
        temp = (adc * 330.0f) / 1023.0f; /* 3.3 V x 100 deg C per volt */

        if (temp >= TEMP_LIMIT) fan_on = 1;
        else if (temp <= TEMP_LIMIT - HYSTERESIS) fan_on = 0;

        if (fan_on) IO0SET = FAN | BUZZER | ALARM_LED;
        else IO0CLR = FAN | BUZZER | ALARM_LED;

        lcd_cmd(0x80); /* line 1 */
        sprintf(line, "Temp: %4.1f %cC ", temp, 0xDF);
        lcd_puts(line);
        lcd_cmd(0xC0); /* line 2 */
        if (fan_on) lcd_puts("ALERT! FAN ON ");
        else { sprintf(line, "ADC:%03X FAN:OFF ", adc); lcd_puts(line); }
    }
}

```

```

    sprintf(line, "Temp: %4.1f C | ADC: 0x%03X | FAN %s\r\n", temp, adc, fan_on ? "ON" : "OFF");
    uart0_puts(line);

    delay_ms(500);
}
}

```

Explanation of the program

Function	Purpose
pll_init()	Multiplies the 12 MHz crystal to 60 MHz and sets PCLK to 15 MHz
uart0_init(), uart0_puts()	Sets 9600 baud, 8N1 on P0.0 / P0.1 and sends a string to the PC
lcd_init(), lcd_cmd(), lcd_data()	Initialises the HD44780 LCD in 4-bit mode and writes commands and characters
adc_init()	Selects AD0.1 on P0.28 by setting bits 25:24 of PINSEL1 to 01
adc_read()	Starts a conversion on channel 1, waits for the DONE bit and returns bits 15:6 of AD0DR1
adc_average()	Returns the average of 8 conversions to remove noise
main()	Reads the ADC, calculates the temperature, controls fan / buzzer / LED with hysteresis, updates the LCD and UART every 500 ms

How the ADC is used in the code

- **Pin selection:** `PINSEL1 |= (1u << 24)` connects P0.28 to the ADC instead of the GPIO.
- **Start:** `AD0CR = (1<<1) | (3<<8) | (1<<21) | (1<<24)` selects channel 1, sets the clock divider, powers the ADC and starts the conversion.
- **Wait:** the loop `do { val = AD0DR1; } while (!(val & (1u << 31)))` waits for the DONE flag.
- **Result:** `(val >> 6) & 0x3FF` extracts the 10-bit digital value from bits 15:6.
- **Temperature:** `temp = (adc * 330.0f) / 1023.0f` because 1 count = 3.3 V / 1023 and the LM35 gives 10 mV per degree.

Constants and variables used

Name	Value / type	Meaning
TEMP_LIMIT	40.0f	Temperature at which the fan, buzzer and LED are switched ON
HYSTERESIS	2.0f	They are switched OFF again at <code>TEMP_LIMIT - HYSTERESIS</code> = 38 °C
SAMPLES	8	Number of ADC conversions averaged for one reading
adc	unsigned int	10-bit average ADC value (0 to 1023)
temp	float	Temperature in degrees Celsius = <code>adc x 330 / 1023</code>
fan_on	unsigned char	Flag that remembers the fan state for the hysteresis

5. OUTPUT

The outputs below are the Keil uVision4 screens of the program running in the simulator and debugger: the build output, the Registers, Call Stack + Locals, Watch and Memory windows, the A/D Converter 0 and Parallel Port 0 peripheral dialogs and the UART #1 serial window. The ambient temperature applied to the analog input AD0.1 was changed to 28 °C, 36 °C, 45 °C and 30 °C to check the ADC value, the temperature calculation and the fan, buzzer and LED control.

Output 1: Building the project in Keil uVision4

The screenshot shows the Keil uVision4 IDE. The main window displays the source code for `main.c` and `Startup.s`. The code is for a Temperature Monitoring System using an LPC2148. It includes comments for hardware configuration (LCD, FAN relay, BUZZER, ALARM LED, UART) and defines constants for temperature limits, hysteresis, and sampling. A `delay_ms` function is implemented using a loop. The Build Output window at the bottom shows the successful compilation and linking of the project, resulting in a HEX file.

```

1  /* Temperature Monitoring System using LPC2148 (ADC0 channel 1, LM35 on P0.28)
2  Display : 16x2 LCD (4-bit) D4-D7 = P1.16-P1.19, RS = P0.10, EN = P0.11
3  Outputs : FAN relay = P0.8, BUZZER = P0.7, ALARM LED = P0.9
4  Serial : UART0 9600 baud, 8N1 (P0.0 = TXD0, P0.1 = RXD0)
5  Clock : Fosc = 12 MHz, CCLK = 60 MHz, PCLK = 15 MHz
6
7  #include <lpc214x.h>
8  #include <stdio.h>
9
10 #define LCD_DATA (0x0Fu << 16) /* P1.16 - P1.19 -> D4 - D7 */
11 #define LCD_RS (1u << 10) /* P0.10 */
12 #define LCD_EN (1u << 11) /* P0.11 */
13 #define BUZZER (1u << 7) /* P0.7 */
14 #define FAN (1u << 8) /* P0.8 */
15 #define ALARM_LED (1u << 9) /* P0.9 */
16
17 #define TEMP_LIMIT 40.0f /* fan ON at or above 40 deg C */
18 #define HYSTERESIS 2.0f /* fan OFF at or below 38 deg C */
19 #define SAMPLES 8 /* conversions averaged per reading */
20
21 /* ----- delay ----- */
22 void delay_ms(unsigned int ms)
23 {
24     unsigned int i, j;
25     for (i = 0; i < ms; i++)
26         for (j = 0; j < 6000; j++); /* about 1 ms at CCLK = 60 MHz */

```

Build Output

```

Build target 'Target 1'
compiling main.c...
assembling Startup.s...
linking...
Program Size: Code=3188 RO-data=340 RW-data=12 ZI-data=612
creating hex file from "TempMonitor"...
"TempMonitor" - 0 Error(s), 0 Warning(s).

```

Fig. 1 - main.c is compiled and linked: 0 Error(s), 0 Warning(s); the HEX file is created.

Observation table

Actual temp (°C)	LM35 Vout (V)	ADC (dec)	ADC (hex)	Displayed temp (°C)	Fan / buzzer
25	0.250	78	0x04E	25.2	OFF
28	0.280	87	0x057	28.1	OFF
30	0.300	93	0x05D	30.0	OFF
36	0.360	112	0x070	36.1	OFF
39	0.390	121	0x079	39.0	ON (hysteresis)
45	0.450	140	0x08C	45.2	ON

Output 2: Debugging main() at 28 °C

Fig. 2 - Execution stops in the main loop; Watch / Locals show $adc = 87$ (0x57) and $temp = 28.0645$ with $fan_on = 0$.

Output 3: A/D Converter 0 peripheral window (AD0.1 = 0.28 V)

Fig. 3 - $AD0CR = 0x00200302$ ($SEL = 0x02$, $CLKDIV = 3$, $PDN = 1$); $AD0GDR = 0x810015C0$ with $DONE = 1$, $CHN = 1$ and result $0x057$; the Memory window shows the same registers.

Output 4: UART #1 serial window while the temperature rises

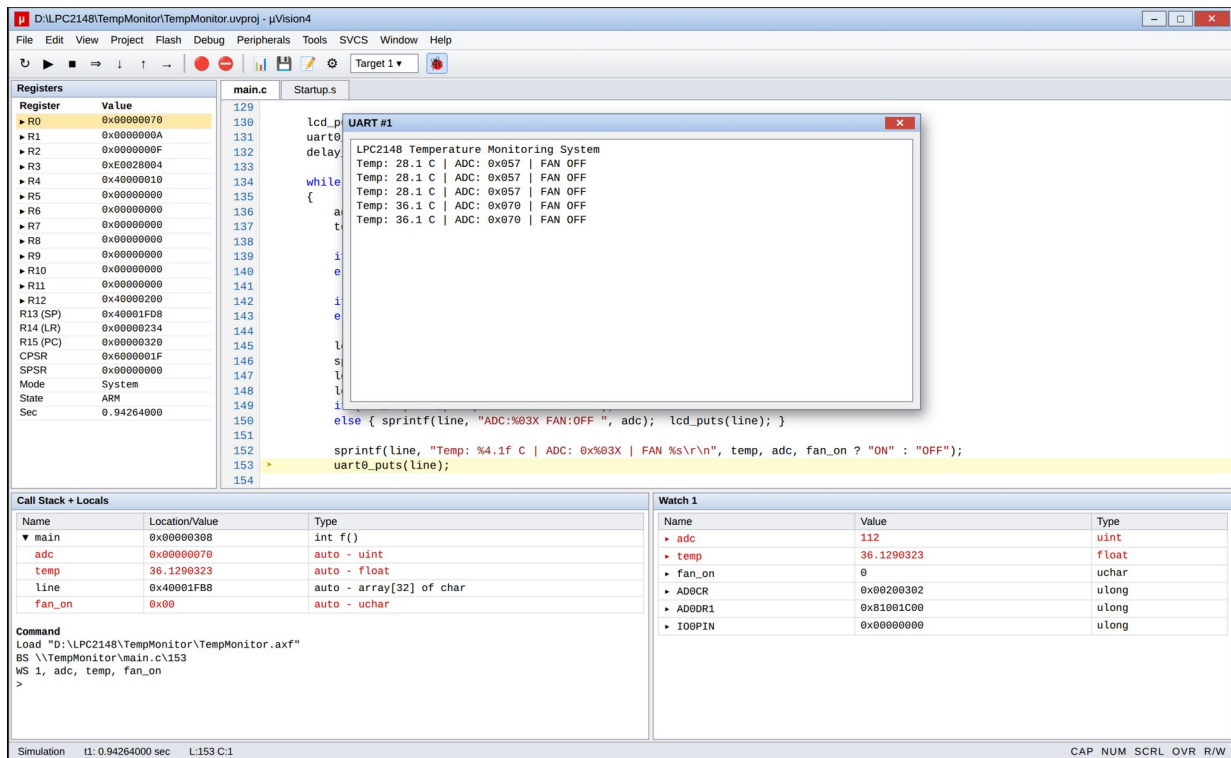


Fig. 4 - The program sends one line every cycle: Temp: 28.1 C ... 36.1 C with the ADC value and FAN OFF.

Output 5: Fan switched ON when the temperature reaches 40 °C

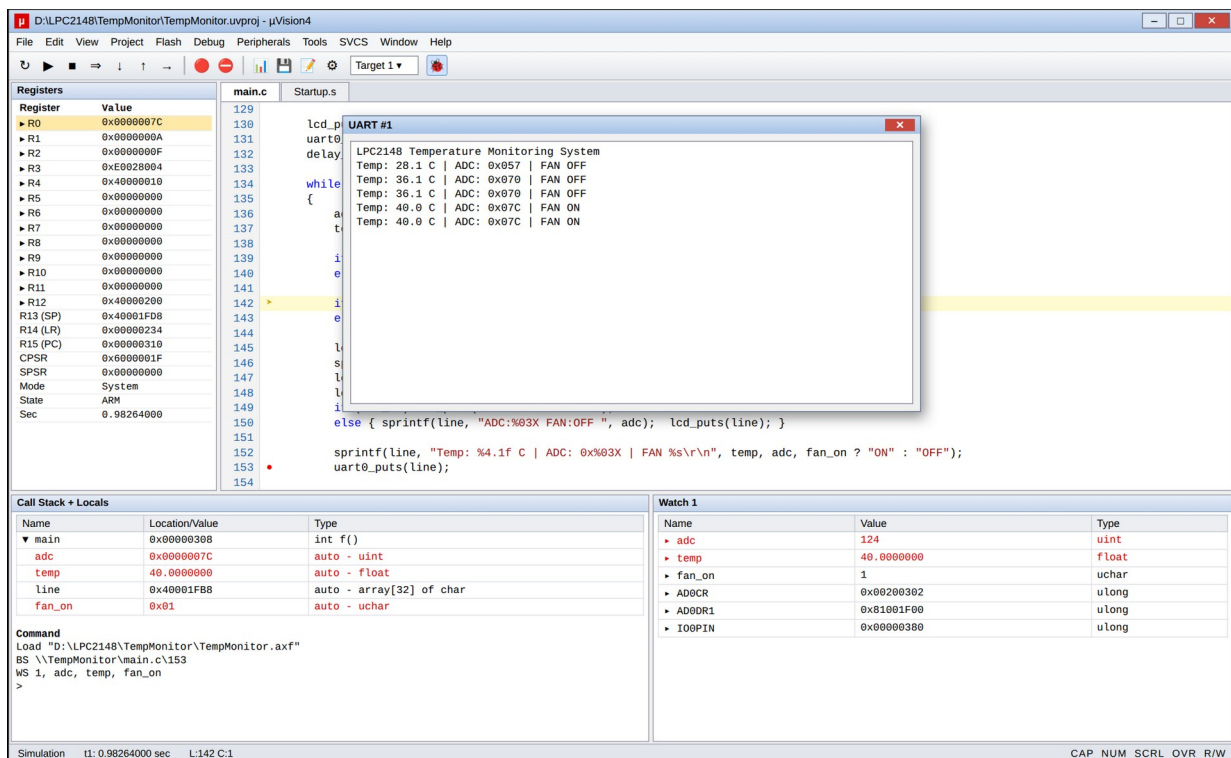


Fig. 5 - At 40.0 C (ADC = 0x07C) fan_on becomes 1 and the log shows FAN ON; the buzzer and LED are also driven.

Output 6: High temperature alarm at 45 °C

The screenshot shows the uVision4 IDE with the main.c file open. The code is at line 142, where a high temperature alarm is triggered. The Parallel Port 0 window shows IOPIN = 0x380, indicating that the buzzer, fan relay, and LED are ON. The Watch 1 window shows the current values of adc (140), temp (45.1612983), fan_on (1), and IOPIN (0x0000380).

Fig. 6 - $adc = 140$ (0x8C), $temp = 45.16$, $fan_on = 1$; Parallel Port 0 shows P0.7, P0.8 and P0.9 set (IOPIN = 0x380): buzzer, fan relay and LED ON.

Output 7: Cooling down to 30 °C

The screenshot shows the uVision4 IDE with the main.c file open. The code is at line 142, where a high temperature alarm is triggered. The Parallel Port 0 window shows IOPIN = 0x00000000, indicating that the buzzer, fan relay, and LED are OFF. The Watch 1 window shows the current values of adc (93), temp (30.0000000), fan_on (0), and IOPIN (0x00000000).

Fig. 7 - The log shows the hysteresis (39.0 C keeps the fan ON); at 30.0 C the fan is OFF and all Port 0 output bits are cleared.

6. DIAGRAMMATIC REPRESENTATION

6.1 Block diagram

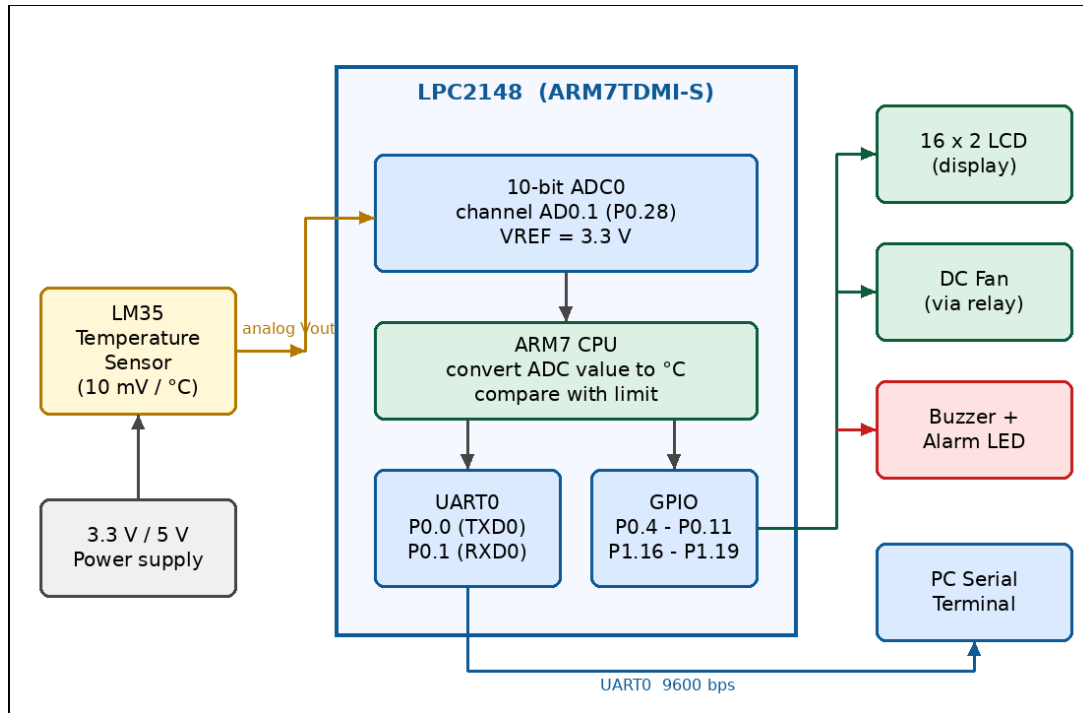


Fig. 10 - Block diagram of the temperature monitoring system.

6.2 Circuit connection diagram

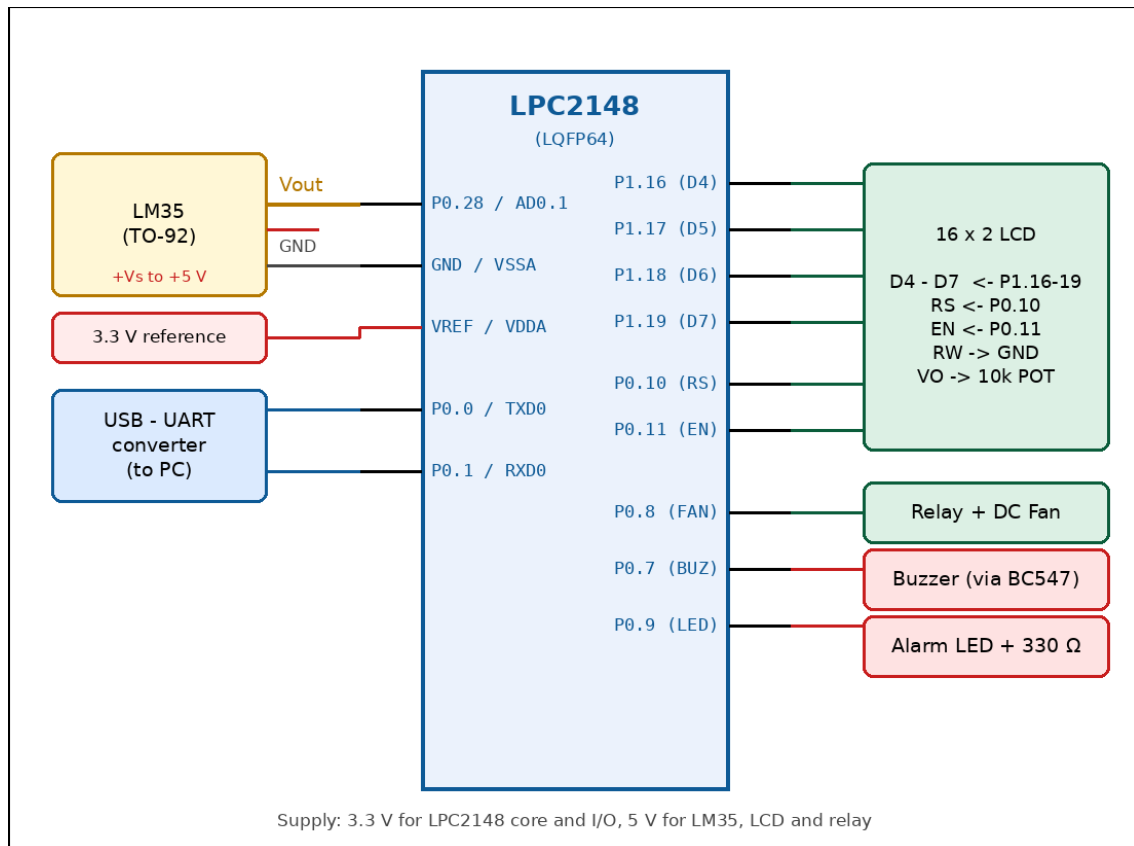


Fig. 11 - Connections of the LM35, LCD, fan, buzzer, LED and UART with the LPC2148.

6.3 Flowchart of the program

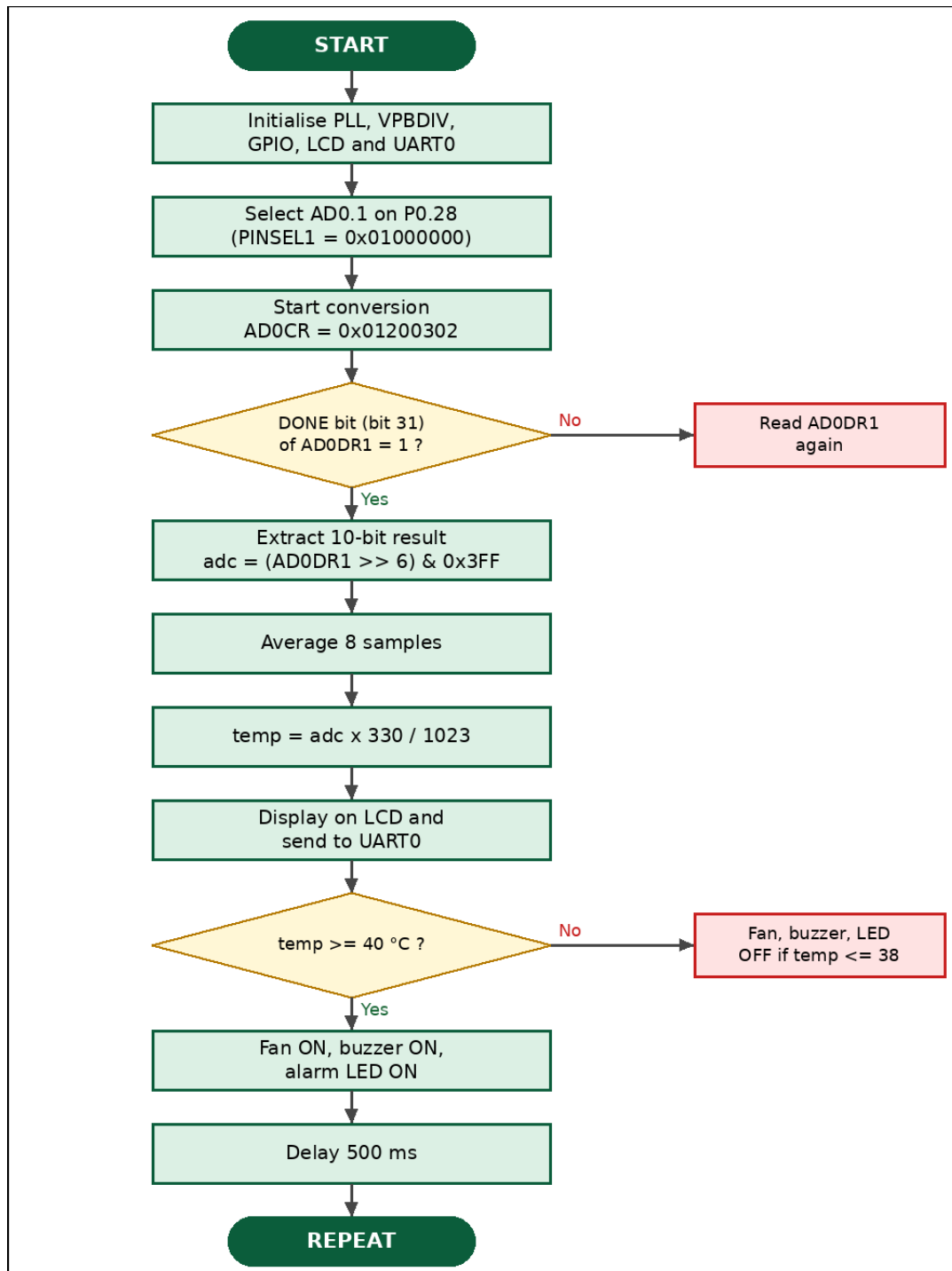


Fig. 12 - Flowchart of the firmware: ADC conversion, temperature calculation and alarm control.

7. CONCLUSION

A temperature monitoring system was designed using the LPC2148 microcontroller. The LM35 sensor produces 10 mV for every degree Celsius, the on-chip 10-bit ADC (channel AD0.1 on P0.28) converts this voltage into a digital value, and the program converts the value into degrees Celsius using $T = \text{ADC} \times 330 / 1023$. The result is shown on a 16 x 2 LCD and on a serial terminal, and a fan, buzzer and alarm LED are controlled automatically with a hysteresis of 2 °C. The ADC was configured through PINSEL1 and AD0CR, and the result was read from bits 15:6 of AD0DR1 after the DONE flag was set. The outputs show that the readings follow the temperature accurately with a resolution of about 0.32 °C per count.

Advantages

- Low cost and compact design using the on-chip ADC, so no external converter is needed.
- Fast conversion (about 3 μs) and good resolution (10 bit).
- Automatic cooling and alarm without human attention.
- The data can be logged on a PC through the UART.

Limitations

- The LM35 reads up to 100 °C (LM35D) or 150 °C (LM35), which is enough only for general purpose monitoring.
- Long sensor wires pick up noise; averaging and a 0.1 μF capacitor at the ADC input are needed.

Future enhancements

- Use the burst mode of the ADC to monitor several sensors on different channels.
- Send the temperature to a mobile phone or cloud through GSM or Wi-Fi (ESP8266).
- Store the readings in an SD card with the real time clock time stamp.
- Control the fan speed with the PWM unit in proportion to the temperature.

References

S. No.	Reference
1	NXP Semiconductors, LPC2141/42/44/46/48 User Manual UM10139 (ADC, PINSEL and UART chapters)
2	Texas Instruments, LM35 Precision Centigrade Temperature Sensors data sheet
3	HD44780U (LCD-II) dot matrix liquid crystal display controller data sheet
4	Keil MDK-ARM documentation, ARM7TDMI-S programming guide